

Exercise 3: Stored Procedures

Aim

To understand and implement stored procedures in MySQL for banking operations such as processing monthly interest, updating employee bonuses, and transferring funds between accounts.

```
CREATE DATABASE BankDB;
```

```
USE BankDB;
```

```
CREATE TABLE Customer(  
    CustomerID INT PRIMARY KEY,  
    CustomerName varchar(50),  
    Age INT,  
    Balance decimal(10,2),  
    LoanInterestRate decimal(5,2),  
    IsVIP boolean  
);
```

```
CREATE TABLE Loans(  
    LoanID INT PRIMARY KEY,  
    CustomerID INT,  
    LoanAmount DECIMAL(10,2),  
    LoanDueDate DATE,  
    FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID)  
);
```

Insert into Customer Values

```
(101,'Krishna',65,15000,9.5,FALSE),  
(102,'Rahul',45,8000,10.0,FALSE),  
(103,'Priya',70,20000,8.5,FALSE),  
(104,'Arun',35,12000,11.0,FALSE),  
(105,'Sneha',62,5000,9.0,FALSE);
```

INSERT INTO Loans VALUES

```
(1,101,500000,'2026-07-15'),  
(2,102,300000,'2026-09-10'),  
(3,103,400000,'2026-07-20'),  
(4,104,250000,'2026-08-25'),  
(5,105,350000,'2026-07-10');
```

CREATE TABLE Account(

AccountID INT PRIMARY KEY,

CustomerID INT,

AccountType VARCHAR(20),

Balance DECIMAL(10,2),

FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID)

);

INSERT INTO Account VALUES

```
(1,101,'Savings',50000),
```

```
(2,102,'Savings',30000),  
(3,103,'Savings',15000),  
(4,104,'Current',40000),  
(5,105,'Savings',25000);
```

```
CREATE TABLE Employee(  
    EmployeeID INT PRIMARY KEY,  
    EmployeeName VARCHAR(50),  
    Salary DECIMAL(10,2)  
);
```

```
UPDATE Employee  
SET Department='HR'  
WHERE EmployeeID=201;
```

```
UPDATE Employee  
SET Department='Finance'  
WHERE EmployeeID=202;
```

```
UPDATE Employee  
SET Department='Finance'  
WHERE EmployeeID=203;
```

--**Question:** Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

```
SELECT * FROM Account;
```

	AccountID	CustomerID	AccountType	Balance
▶	1	101	Savings	50000.00
	2	102	Savings	30000.00
	3	103	Savings	15000.00
	4	104	Current	40000.00
	5	105	Savings	25000.00
•	NULL	NULL	NULL	NULL

```
DELIMITER $$
```

```
CREATE PROCEDURE ProcessMonthlyInterest()
```

```
BEGIN
```

```
UPDATE Account
```

```
SET Balance = Balance + (Balance*0.01);
```

```
END $$
```

```
DELIMITER ;
```

```
CALL ProcessMonthlyInterest();
```

SELECT * FROM Account;

	AccountID	CustomerID	AccountType	Balance
▶	1	101	Savings	50500.00
	2	102	Savings	30300.00
	3	103	Savings	15150.00
	4	104	Current	40400.00
	5	105	Savings	25250.00
*	NULL	NULL	NULL	NULL

--**Question:** Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

DELIMITER \$\$

CREATE PROCEDURE UpdateEmployeeBonus(

IN Dept VARCHAR(30),

IN BonusPercent DECIMAL(5,2)

)

BEGIN

UPDATE Employee

SET Salary = Salary + (Salary * BonusPercent / 100)

WHERE Department = Dept;

END \$\$

DELIMITER ;

SELECT * FROM EMPLOYEE;

	EmployeeID	EmployeeName	Salary	Department
▶	201	Ravi	60000.00	HR
	202	Anita	60000.00	Finance
	203	John	45000.00	Finance
*	NULL	NULL	NULL	NULL

CALL UpdateEmployeeBonus('Finance',10);

SELECT * FROM Employee;

	EmployeeID	EmployeeName	Salary	Department
▶	201	Ravi	60000.00	HR
	202	Anita	66000.00	Finance
	203	John	49500.00	Finance
*	NULL	NULL	NULL	NULL

--**Question:** Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

DELIMITER \$\$

CREATE PROCEDURE TransferFunds(

IN FromAcc INT,

IN ToAcc INT,

IN Amount DECIMAL(10,2)

)

BEGIN

DECLARE CurrentBalance DECIMAL(10,2);

SELECT Balance

INTO CurrentBalance

FROM Account

WHERE AccountID = FromAcc;

IF CurrentBalance >= Amount THEN

UPDATE Account

SET Balance = Balance - Amount

WHERE AccountID = FromAcc;

UPDATE Account

SET Balance = Balance + Amount

WHERE AccountID = ToAcc;

SELECT 'Transfer Successful' AS Message;

ELSE

SELECT 'Insufficient Balance' AS Message;

END IF;

END \$\$

DELIMITER ;

SELECT * FROM Account;

	AccountID	CustomerID	AccountType	Balance
▶	1	101	Savings	50500.00
	2	102	Savings	30300.00
	3	103	Savings	15150.00
	4	104	Current	40400.00
	5	105	Savings	25250.00
*	NULL	NULL	NULL	NULL

CALL TransferFunds(1,2,5000);

	Message
▶	Transfer Successful

SELECT * FROM Account;

	AccountID	CustomerID	AccountType	Balance
▶	1	101	Savings	45500.00
	2	102	Savings	35300.00
	3	103	Savings	15150.00
	4	104	Current	40400.00
	5	105	Savings	25250.00
*	NULL	NULL	NULL	NULL